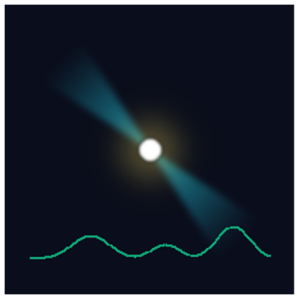
DSES Radio Astronomy Workbench

Release Workflow



Version 1.7.0

Richard M Hambly (K0GD)

DSES

November 2026

Contents

1. Development environment 3
2. Source layout 4
3. Distribution host 5
4. Routine release workflow 6
    4.1 Bump the version 6
    4.2 Commit the source changes 6
    4.3 Rebuild the user-facing install guide 6
    4.4 Rebuild this document (optional) 7
    4.5 Build the release zip 7
    4.6 Upload to the server 7
    4.7 Update the manifest 9
    4.8 Tag the release commit 9
    4.9 Refresh the public GitHub mirror 10
    4.10 Done 10
5. The update manifest 10
    5.1 Manifest format 10
    5.2 Server side: writing the manifest 11
    5.3 Behavior notes 11
    5.4 Disabling and rolling back 12
6. Testing a release 12
    6.1 Verify the manifest fetch and version compare 12
    6.2 Verify the notification dialog renders 12
    6.3 Verify the download URL 13
7. Reference 13
    7.1 Files the release script bundles 13
    7.2 Where the SigMF sample comes from 14
    7.3 Documentation toolchain 15
    7.4 Multi-radio architecture 16
    7.5 Memory / notes worth knowing 19

DSES Radio Astronomy Workbench — Release Workflow

Version 1.0.0 Author: Richard M Hambly (K0GD) — k0gd@cnssys.com Audience: developer only — not shipped to recipients.

This document is for the developer (you) cutting and publishing new releases. It complements `DSES_Radio_Astronomy_Workbench_Installation.pdf`, which is the user-facing install guide. Topics covered here:

- How the project is laid out and what the build tools need.
- The end-to-end routine for shipping a new release.
- How the in-app auto-update notification works and what to put on the server.
- How to test a release before announcing it.
- Reference: distribution host, file naming conventions, source-code constants.

1. Development environment

All development happens in the project-local conda environment at `./.conda`. CLAUDE.md captures the project rules:

- This env is created from `environment.yml` with `conda env create --prefix ./.conda -f environment.yml`.
- Dependency changes go in `environment.yml`. No global installs, no shell rc edits.
- The Qt binding is **PySide6** (commercial Qt license + LGPL friendlier for distribution). Never import from `PyQt5` / `PyQt6`.

The runtime is **Radioconda** (installed by each end user). The `./.conda` env adds what's needed for development plus the doc-build chain — `python-docx` to render the styled DOCX, and **LibreOffice** (a system install, not conda) to convert it to PDF on macOS/Linux (Windows uses Microsoft Word via `pywin32`).

Runtime dependencies beyond stock Radioconda: the app imports `PySide6`, `pyqtgraph`, and `scipy`, which Radioconda does **not** bundle (it ships PyQt5 for GNU Radio's own qtgui). End users install them once with `conda install -c conda-forge pyside6 pyqtgraph scipy` — this is install-guide §2.4, and the launchers preflight-check for them and print that command if missing. If you ever add another third-party import to the app, add it to that list in: install guide §2.4, the launcher preflight checks (`launcher.sh` / `launcher.ps1`), and the §6 troubleshooting entry.

To rebuild `./.conda` from scratch:

```
conda env create --prefix ./.conda -f environment.yml
```

or to update an existing one:

```
conda env update --prefix ./.conda -f environment.yml --prune
```

2. Source layout

Path	Purpose
`dses_workbench.py`	The application. Single hand-polished Python file. `APP_VERSION` near the top is the source of truth for the release version. (Was `dses_spectrum_analyzer.py` through 1.3.4.)
`dses_spectrum_analyzer.py`	Launch shim under the pre-1.4.0 module name — just runs `dses_workbench.py`. Ships so an in-place update overwrites the stale old application file.
`launcher.bat` / **`.ps1`**	Windows entry point + launcher logic (finds Radioconda).
`launcher.sh`	Linux / macOS launcher.
`install-shortcut.ps1`	Windows desktop / Start-menu shortcut installer.
`dses-workbench.desktop`	Linux desktop entry template.
`icons/dses_workbench.{ico,png,icns}`	Windows / Linux / macOS icons.
`icons/generate-icon.py`	Source for the icon. Re-run only if you change the design.
`sample.sigmf-data` + **`sample.sigmf-meta`**	Bundled playback sample. Loaded by the app when no B210 is attached.
`LICENSE`	GPL-3.0 full text + project copyright.
`environment.yml`	Conda env spec (dev).
`Installing.md`	Source for the user-facing install guide.
`Release_Workflow.md`	This document.
`build_doc.py`	Builds the styled `.pdf` from a Markdown source (rendering a throwaway temp DOCX on the way). CLI-parameterized (`--title`, `--subtitle`, `--pdf`; `--docx` keeps the intermediate).
`make-release.ps1` / **`.sh`**	Stages the runtime files into `dist/dses-workbench-<version>/` and zips them.
`CLAUDE.md`	Project ground rules for Claude Code sessions. Not shipped.

Not part of the release bundle: `.conda/`, `.git/`, `.vscode/`, `dist/`, `CLAUDE.md`, `Installing.md`, `Release_Workflow.md`, `build_doc.py`, `make-release.{ps1,sh}`, `icons/generate-icon.py`.

3. Distribution host

All distribution goes through:

```
URL:        https://gpstime.com/sw_distribution/dses-workbench/

Filesystem: /var/www/html/sw_distribution/dses-workbench/   (on gpstime.com)
```

Upload releases straight into that filesystem directory (Apache/nginx docroot is `/var/www/html`). A common mistake is to scp into your home directory (e.g. `~/dses_workbench_dist`) — files there are NOT web-served and the URL 404s. Always `mv` them into `/var/www/html/sw_distribution/dses-workbench/` and `chmod 644`.

That single directory holds:

- `manifest.json` — the version manifest the running app polls.
- `dses-workbench-<version>.zip` — one zip per published release. Keep at least the current + previous version online.
- `dses-workbench-<version>.sha256` — optional SHA-256 next to each zip so recipients can verify their download.
- `DSES_Radio_Astronomy_Workbench_Installation.pdf` — latest install guide, separate from the zips, for users who want to read before downloading the bundle. (Also bundled inside each zip; the standalone copy is the "read first" link — see §4.6.)
- `.htaccess` — enables directory listing for this folder (`Options +Indexes` + `FancyIndexing`, with `IndexIgnore .htaccess`). Without it Apache returns 403 on the bare directory. Leave it in place; it's why users can browse the folder as well as use direct file links.

Legacy folder `sw_distribution/b210_sa/` — keep it, and keep its manifest pointing at the current release. Every release through 1.3.4 lived there under the `dses-spectrum-analyzer-<version>.zip` name, and every install of those versions has `…/b210_sa/manifest.json` persisted in its `settings.ini`. The updater follows whatever `download_url` a manifest names and accepts any top-level folder name in the zip, so an old install migrates cleanly as long as the old manifest advertises the new zip. Therefore each release writes TWO manifests with identical content (§5.2): the real one in `dses-workbench/` and the pointer in `b210_sa/`. Once an install has run 1.4.0 or later, its stored URL is migrated to the new folder by the app itself. The old zips can stay where they are; nothing needs to be copied or renamed.

There is no per-OS variant — one zip works on Windows, Linux (any current distro), macOS Intel, and macOS Apple Silicon. The OS-specific launchers ride along inside the bundle.

4. Routine release workflow

When you're ready to ship a new version:

4.1 Bump the version

Edit `APP_VERSION` near the top of `dses_workbench.py`. Use semantic versioning: `MAJOR.MINOR.PATCH`. Bug fixes only → bump PATCH; new features → bump MINOR; breaking changes → bump MAJOR.

This is enforced from **1.5.0** (Rick, 2026-09-11): the 1.3.x/1.4.x point releases carried features because the program was still developmental; it is now mostly stable, so the third digit means "bug release" and a feature release moves the second digit.

4.2 Commit the source changes

Note the version in the commit message.

4.3 Rebuild the user-facing install guide

Sync the operating guide first. Installing.md §9 (Appendix C — Operating Guide) is a hand-maintained copy of the in-app Help text (`HELP_TEXT_HTML` in `dses_workbench.py`). If you changed any control or Help wording this release, update §9 to match before rebuilding the PDF. The in-app Help is authoritative; §9 just mirrors it for the printable guide.

Bump the §3 download link. Installing.md §3 has a direct, version-stamped download URL (`…/dses-workbench-<version>.zip`). Update that version to the release you're publishing before rebuilding the PDF. (The folder is also browsable — see §3 of this document — so a slightly stale link isn't fatal, but keep it current.)

```
./.conda/bin/python build_doc.py
```

That reads `Installing.md` and produces `DSES_Radio_Astronomy_Workbench_Installation.pdf` (the deliverable). The styled DOCX it renders on the way is written to a temp file and removed automatically; pass `--docx <path>` if you want to keep it for a spot-check. The cover's version line comes from `APP_VERSION` unless you pass `--version`.

On Windows the PDF is made by Acrobat Distiller, and only Distiller: Word prints PostScript to a file and `acrodist.exe` distills it, so the house fonts land as named subsets. The attempt is bounded (a crashed font-capture helper is killed, the job cancelled) and when it fails the build stops with the diagnosis instead of printing through another engine (since 1.7.0; the earlier automatic fallback to "Microsoft Print to PDF" shipped degraded PDFs unnoticed). The usual cause is the Adobe PDF printer's option "Rely on system fonts only; do not use document fonts" being ticked again — untick it in the printer's Printing preferences and rebuild. `DSES_PDF_ENGINE=msprint|pdfmaker` is an explicit opt-in for a one-off and is not release quality. Word's own PDF export is never used because it rasterizes the house fonts. If any print or save dialog appears

during the build, the script is at fault (Word bound without its type library) — fix it, do not click through. Check the result before shipping it:

```
python tools/verify_pdf.py DSES_Radio_Astronomy_Workbench_Installation.pdf
"Version <version>"
```

That prints pages, extractable characters, and every embedded font, and fails if the text layer or the version stamp is missing (needs `pypdf`).

Skip this step if you didn't change install-relevant behavior, but err on the side of rebuilding so the version stamps inside the PDF stay current.

4.4 Rebuild this document (optional)

```
./.conda/bin/python build_doc.py Release_Workflow.md \

    --pdf DSES_Radio_Astronomy_Workbench_Release_Workflow.pdf \

    --subtitle "Release Workflow"
```

Only if you've edited this file. It doesn't ship.

4.5 Build the release zip

```
Windows:  .\make-release.ps1

Unix:     ./make-release.sh
```

The script reads `APP_VERSION`, creates `dist\dses-workbench-<version>\` with the runtime files, and zips it to `dist\dses-workbench-<version>.zip`. The staging directory is kept so you can inspect the contents before publishing.

Launcher check (Windows cuts). Before publishing, run `.conda\python.exe test_launcher.py`. It starts the launcher from a folder whose name contains spaces, with stand-in programs, and fails if the program does not receive its full path or if a program that stops at once is not reported. Through 1.6.0 the launcher could not start from a path with a space; this test exists so that cannot return. On Linux, or from Windows through WSL, `python3 test_desktop_entry.py` checks the desktop-entry template the same way.

4.6 Upload to the server

Upload **into the directory** `/var/www/html/sw_distribution/dses-workbench/` on gpstime.com. Two files go up each release:

- `dses-workbench-<version>.zip` — the bundle.

- `DSES_Radio_Astronomy_Workbench_Installation.pdf` — standalone install guide (bundled in the zip too, but the standalone copy is the "read before you download the 200+ MB bundle" link). Overwrite it every release so the public guide stays in sync.

Method A — FileZilla (SFTP), the usual method.

1. Connect to gpstime.com over **SFTP** (protocol "SFTP - SSH File Transfer Protocol", port 22), using your SSH login.
2. Navigate the *remote* pane to `/var/www/html/sw_distribution/dses-workbench/`. If you land in your home directory and can't see `/var/www`, type the path into FileZilla's "Remote site:" box and press Enter.
3. Drag the zip and the PDF over.
4. **Set permissions to 644** so the web server can read them: right-click each uploaded file → **File permissions…** → set the numeric value to `644` (or tick read for owner/group/public, write for owner only). Without this the file may exist but the URL returns 403.

> If FileZilla can't write into `/var/www/html/...` (permission denied), the SSH account doesn't own that path. Either have the host grant write access to that directory, or upload to your home dir and ask whoever has `sudo` to move the files into place. This is a server-account setup issue, not a FileZilla bug.

Method B — scp (command line), if you prefer.

Run it as a **single line** (scp takes multiple source files before the destination). Do NOT break it across lines with `^` — that's a CMD continuation char and fails in PowerShell (which uses a backtick); the safe choice is one line:

```
scp dist\dses-workbench-<version>.zip
DSES_Radio_Astronomy_Workbench_Installation.pdf
rick@gpstime.com:/var/www/html/sw_distribution/dses-workbench/
```

The SSH username is `rick` (so `rick@gpstime.com`), not an email. Common scp gotchas: the path after the colon is absolute (leading `/`); on Windows, run scp from PowerShell (OpenSSH client) or Git Bash, not the conda prompt.

Ownership note: `/var/www/html/sw_distribution/dses-workbench/` and its contents are owned by `rick:rick`, so `rick` can overwrite the zip/PDF/manifest directly on each release — no `sudo` needed. If you ever see *"dest open … Permission denied"* on an overwrite, a file in there reverted to root ownership (e.g. something dropped in via `sudo`); fix it once with `sudo chown -R rick:rick /var/www/html/sw_distribution/dses-workbench/`.

After uploading (either method), generate the checksum and verify the URLs. Easiest from an SSH session:

```
cd /var/www/html/sw_distribution/dses-workbench/

sha256sum dses-workbench-<version>.zip > dses-workbench-<version>.sha256

chmod 644 dses-workbench-<version>.{zip,sha256}
DSES_Radio_Astronomy_Workbench_Installation.pdf

curl -sI https://gpstime.com/sw_distribution/dses-workbench/dses-workbench-
<version>.zip | head -1

curl -sI https://gpstime.com/sw_distribution/dses-
workbench/DSES_Radio_Astronomy_Workbench_Installation.pdf | head -1

# expect "HTTP/2 200" or "HTTP/1.1 200 OK"
```

4.7 Update the manifest

Edit `manifest.json` on the server with the new `latest_version`, `download_url`, and `release_notes`, then copy it over the legacy `b210_sa/manifest.json` pointer as well (§3, §5.2). Format and behavior in §5 below.

4.8 Tag the release commit

Once the live verification passes, tag the commit the zip was built from (the one carrying the new `APP_VERSION`, not a later STATE/notes commit) and push the tag, so every published version is recoverable from git by name:

```
git tag -a v<version> <release-commit> -m "DSES Radio Astronomy Workbench
<version> - <one-line summary> (published <date>, zip sha256 <first 8 hex>...)"

git push origin v<version>
```

Convention started at v1.4.0 (2026-09-08); releases before that have no tags. Use the annotated form (`-a`) so the tag records who cut it and when.

4.9 Refresh the public GitHub mirror

The development history is mirrored read-only at <https://github.com/dses-science/dses-workbench> in the DSES GitHub organization (same arrangement as the EVE modem's `dses-science/eve-modem`; mirrored since 2026-09-22, in the organization since 2026-09-23). The NAS bare repo stays the master; the mirror carries `main` and every release tag with the private working notes (`CLAUDE.md`) removed from every commit by `git filter-repo`, and co-author trailers removed from every commit and tag message. After the release commit and tag are pushed to origin:

```
bash tools/publish_github.sh
```

The rewrite is deterministic (the mirror's commit ids stay stable across runs) and the script refuses to run while local `main` is ahead of `origin/main`. Needs `git filter-repo` and a GitHub login (`gh auth login`). Run it after any push to origin, not only at release time. Releases themselves are never published from GitHub — the zip, guide and manifest live on gpstime (§4.6–4.7).

4.10 Done

Existing users running the previous release (with auto-update enabled and a working manifest URL configured) see the notification on next launch — within 24 hours of relaunch, since the program throttles checks to one per 24 hours.

5. The update manifest

The program polls a single JSON file (the "manifest") to learn about new releases. The URL is baked into the default settings as:

```
https://gpstime.com/sw_distribution/dses-workbench/manifest.json
```

End users can override it in their own `settings.ini` (`manifest_url =` under `[updates]`), e.g. to point at a staging copy for testing before publishing to all users.

5.1 Manifest format

JSON object with these fields:

```
{
  "latest_version":  "1.0.1",
  "download_url":    "https://gpstime.com/sw_distribution/dses-workbench/dses-workbench-1.0.1.zip",
  "release_notes":   "Added auto-update check.\nFixed recording freeze at 25 MHz.\n"
```

```
}
```

- **`latest_version`** — required. Dotted version string. The program compares this to its own `APP_VERSION` using tuple-of-ints semantics.
- **`download_url`** — required (unless you want the Open button greyed out). The dialog opens this URL in the user's browser.
- **`release_notes`** — optional. Plain text rendered in a scrollable panel. Use `\n` for line breaks.

5.2 Server side: writing the manifest

From your SSH session, after uploading the new zip:

```
cat > /var/www/html/sw_distribution/dses-workbench/manifest.json << 'EOF'

{

  "latest_version": "1.0.1",

  "download_url": "https://gpstime.com/sw_distribution/dses-workbench/dses-
workbench-1.0.1.zip",

  "release_notes": "Added X.\nFixed Y.\n"

}

EOF

chmod 644 /var/www/html/sw_distribution/dses-workbench/manifest.json
```

The single-quoted `<< 'EOF'` is important so the shell doesn't expand `$`/backslashes inside the JSON.

Then copy the same manifest into the legacy folder so installs that still poll the pre-1.4.0 URL find the release (see §3):

```
cp /var/www/html/sw_distribution/dses-workbench/manifest.json
/var/www/html/sw_distribution/b210_sa/manifest.json

chmod 644 /var/www/html/sw_distribution/b210_sa/manifest.json

curl -sS https://gpstime.com/sw_distribution/b210_sa/manifest.json | head -3   #
must show the new download_url
```

Do not skip this: an old install that reads a stale `b210_sa` manifest simply never learns about the update.

5.3 Behavior notes

- The auto-check runs at app launch on a background thread; failures are silent. Manual `Help → Check for Updates…` shows an error dialog on failure.

- Throttled to one fetch per 24 hours per user. `last_check_iso` in their `settings.ini` records the last successful check.
- If the user clicks **Skip this version**, the version is saved to `dismissed_version` and they aren't shown the dialog again until you publish a strictly newer one. Bug-fix releases for the same `latest_version` don't re-prompt — bump the version if you want to re-notify.
- TLS cert: Python's `urllib.request` verifies by default. `gpstime.com` needs a normal (Let's Encrypt / commercial) cert. Self-signed will fail silently.
- Server doesn't need to serve `application/json` MIME type, but it's polite.

5.4 Disabling and rolling back

- To **revoke** a release (security issue, etc.): lower `latest_version` in the manifest back to the previous good version. Running clients on the bad version won't be told to update, but you'd still have to ask them out-of-band to roll back.
- To **disable** auto-update entirely for everyone: serve an empty manifest URL (or a 404). Users will silently fail the check at launch.
- To **disable** for a single user: have them set `auto_check = false` under `[updates]` in their `settings.ini`, or set `manifest_url =` (empty).

6. Testing a release

6.1 Verify the manifest fetch and version compare

Before announcing a release, sanity-check the manifest with the same-version test — running app at v1.0.0, manifest also says v1.0.0:

```
curl -sS https://gpstime.com/sw_distribution/dses-workbench/manifest.json

# expected: the JSON you uploaded
```

In the running program, **Help → Check for Updates…** should show "You're running the latest version (1.0.0). Manifest reports latest = 1.0.0." That confirms TLS handshake + HTTP fetch + JSON parse + version compare all work.

6.2 Verify the notification dialog renders

Temporarily edit `manifest.json` on the server to advertise a higher version than what's running. Use **Help → Check for Updates…** (auto-check is debounced — wipe `last_check_iso` in `settings.ini` to retrigger that path). The non-modal update dialog should appear with the release notes you wrote.

Don't forget to flip the manifest back to the real `latest_version` before walking away, or v1.0.0 users will see a fake notification.

6.3 Verify the download URL

```
curl -sI https://gpstime.com/sw_distribution/dses-workbench/dses-workbench-
<version>.zip | head -1
```

Should be `HTTP/... 200`. If not, the **Open Download Page** button in the dialog will 404 in users' browsers.

7. Reference

7.1 Files the release script bundles

`make-release.ps1` (Windows) and `make-release.sh` (Unix) produce identical bundles. Contents:

```
dses-workbench-<version>/
├── dses_workbench.py
├── dses_spectrum_analyzer.py    ← launch shim under the pre-1.4.0 name
├── sigproc_fil.py               ← shared .fil core (app + iq_to_fil)
├── ezra_txt.py                  ← ezRA drift-scan writer
├── fold_analysis.py             ← auto post-processing pipeline
├── fold_pdf.py                  ← self-contained fold PDFs
├── iq_to_fil.py
├── updater.py
├── LICENSE
├── launcher.bat
├── launcher.ps1
├── launcher.sh
├── install-shortcut.ps1
├── install-shortcut.command
├── dses-workbench.desktop
├── icons/dses_workbench.ico
├── icons/dses_workbench.png
├── icons/dses_workbench.icns
```

```
├── presto/presto_bridge.py      ← WSL/native PRESTO bridge (Analyze/Quick look)
├── presto/build_presto*.sh      ← PRESTO install recipes (WSL + macOS)
├── presto/extend_ut1.sh, README.md, par/
├── sdrplay/sdrPlaySupport.dll   ← pre-built SoapySDRPlay3 module (Windows)
├── sdrplay/README.txt           ← what it is + ABI it was built against
├── environment.yml
├── DSES_Radio_Astronomy_Workbench_Installation.pdf
├── sample.sigmf-data
└── sample.sigmf-meta
```

IMPORTANT — when the app grows a new local module (a new top-level `import <module>` of a repo file), add it to BOTH `make-release.ps1` and `make-release.sh` ship lists. Both scripts now run a **local-import completeness check** at stage time and refuse to build a zip whose staged Python imports a repo module that isn't staged — that guard exists because the first published 1.1.8 zip shipped without `ezra_txt.py` and died at startup with `ModuleNotFoundError` on every install that took the update (caught by Rick's production install within the hour; zip replaced in place same day).

The `sdrplay/` payload is sourced from `vendor/windows/` in the repo. SDRplay users on Windows copy the DLL into Radioconda per Installing.md §3A.2; everyone else ignores it (Linux/macOS get the module from their package manager).

`Release_Workflow.md`, the matching PDF, `CLAUDE.md`, `Installing.md`, `build_doc.py`, the `make-release.*` scripts themselves, the conda env, `.git`, and IDE configs are all excluded by name.

7.2 Where the SigMF sample comes from

The bundled `sample.sigmf-{data,meta}` is a real B210 recording at 10 MHz centered on 408 MHz (the pulsar preset). It loops in playback mode and lets users with no radio attached see the GUI work — useful for training.

It is shipped as a **2-second clip** (cf32 @ 10 MS/s = 160 MB), trimmed from the original ~7.2-second capture to keep the zip small (~60 MB vs ~223 MB). The full original is kept locally as `sample_full.sigmf-data` (git-ignored, not committed) so you can re-trim to a different length. Two seconds was judged a good balance: the spectrum looks identical at any length, and the waterfall's loop repetition was acceptable for demo/training.

To re-trim to a different length (cf32 @ 10 MS/s is 80 MB per second, so bytes = seconds × 80,000,000 rounded to a multiple of 8):

```
head -c 160000000 sample_full.sigmf-data > sample.sigmf-data   # 2.0 s
```

To replace it with a different recording entirely:

1. In live mode, hit **Record** and let it run for a few seconds.
2. Stop recording. The new file lands in `~/Documents/DSES_SA_Recordings/DSES_Workbench_<timestamp>.sigmf-{data,meta}`.
3. Rename to `sample.sigmf-data` and `sample.sigmf-meta` and drop next to `dses_workbench.py` in your project root, replacing the existing pair.
4. The next `make-release` picks up the new sample.

7.3 Documentation toolchain

Tool	Used for	Installed via
`python-docx`	Render the styled (temp) DOCX from Markdown.	`environment.yml` (conda-forge)
LibreOffice	Convert the DOCX → PDF on macOS/Linux (headless `soffice`).	system install (macOS: `brew install --cask libreoffice`)
`pywin32`	Windows alternative: drive Microsoft Word via COM for the DOCX → PDF step.	`environment.yml` (conda-forge, Windows-only)

`python-docx` is dev-only and doesn't ship; the runtime (Radioconda) doesn't need it. The PDF step uses whichever engine is present — LibreOffice on macOS/Linux, Word on Windows (see `convert_docx_to_pdf` in `build_doc.py`).

The build script (`build_doc.py`) accepts CLI options so the same code produces both this document and the install guide:

```
./.conda/bin/python build_doc.py [source.md] \
    [--pdf OUT.pdf] [--docx KEEP_DOCX_PATH] \
    [--title "Cover Title"] [--subtitle "Cover Subtitle"]
```

Defaults match the install-guide build, so a bare `build_doc.py` invocation builds `Installing.md` → `DSES_Radio_Astronomy_Workbench_Installation.pdf` with `Installation Guide` as subtitle. The intermediate DOCX is a temp file, removed after the PDF is written (pass `--docx` to keep it).

7.4 Multi-radio architecture

The application drives any of:

- **UHD B200-family**: B200, B210. Via `uhd.usrp_source`, wrapped in `UhdB200Source`.
- **SoapySDR-supported devices**: SDRPlay (RSP1A / RSP1B / RSPduo / RSPdx), RTL-SDR, HackRF, Airspy, Airspy HF+, BladeRF, LimeSDR, PlutoSDR. Via `gnuradio.soapy.source`, wrapped in `SoapyGenericSource`.

Windows DLL loading (the startup block at the top of `dses_workbench.py`). When launched via `python.exe` rather than an activated conda shell, SoapySDR's support modules can't find their vendor DLLs and every one fails with "LoadLibrary() failed". The startup block fixes this on Windows by (1) adding `<sys.prefix>\Library\bin` (and the SDRplay API dir) to the DLL search path so `rtlsdr.dll`, `hackrf.dll`, etc. resolve, and (2) pre-loading Radioconda's own `libusb-1.0.dll` by full path. The libusb pre-load matters because Windows searches `C:\Windows\System32` before PATH, and machines with Zadig / other SDR tools often have an older `System32\libusb-1.0.dll` that lacks symbols the rtlsdr/hackrf/airspy/bladerf modules need — without the pre-load those modules fail with "the specified procedure could not be found" even though `Library\bin` is on PATH. Pre-loading the correct copy first makes every later `LoadLibrary("libusb-1.0.dll")` reuse it (Windows matches loaded modules by base name).

The split-out classes live next to each other in `dses_workbench.py`:

```
RadioSource           — abstract: gr block + set_samp_rate / set_center_freq /
set_gain

  UhdB200Source       — UHD path

  SoapyGenericSource  — gr-soapy path; per-driver capabilities in SOAPY_DEFAULTS
```

`find_all_radios()` = `find_b200_uhd()` + `find_soapy_devices()`. The picker shows them in a single list. Settings hold `(device_driver, device_serial)`.

Per-driver capabilities (sample-rate options, gain range) come from the `SOAPY_DEFAULTS` table at module top. Add a new Soapy backend by listing its driver string and adding an entry there.

SDRPlay-specific: SDRPlay devices need the SDRplay API installer (from sdrplay.com) *and* Pothosware's **SoapySDRPlay3** module installed into Radioconda's SoapySDR modules directory. The module is **not** packaged on conda-forge for any OS.

- **Windows:** we **ship a pre-built `sdrPlaySupport.dll`** in the release zip (`sdrplay/`), sourced from `vendor/windows/` in the repo. Users copy it into `modules0.8\` per Installing.md §3A.2. You only need the build recipe below when the module needs regenerating — see the **Rebuild trigger** note after it.
- **Linux / macOS:** users install the module from their package manager (`apt install soapysdr-module-sdrplay`, `dnf install SoapySDRPlay`, or the Homebrew `pothosware/homebrew-pothos` tap). Nothing to ship.

Rebuild trigger: the shipped DLL is built against **SoapySDR ABI 0.8** (module dir `modules0.8`). If a future Radioconda bumps SoapySDR to 0.9+, the dir becomes `modules0.9` and the 0.8 binary won't load — rebuild with the recipe below and replace `vendor/windows/sdrPlaySupport.dll` (and update its `README.txt`).

TODO — RSPduo: verify tuner mode + gain range when hardware is available. All RSP models share the `sdrplay` driver, so the RSPduo is discovered and driven by the existing `SoapyGenericSource` + `SOAPY_DEFAULTS['sdrplay']` with no code changes expected. Two things to confirm on a real RSPduo (currently at the DSES Haswell, CO site):

- **Tuner mode:** the RSPduo has two tuners (single-tuner / dual-tuner / master-slave). We open one channel with just `driver=sdrplay,serial=...`, which should default to single-tuner (tuner A). Confirm it enumerates as one device and opens cleanly; if it needs an explicit tuner arg, append it to the device-args string in `SoapyGenericSource.__init__` (not a new SDR type).
- **Gain range:** verify the overall gain-reduction range really is 0–48 for the RSPduo (RF gain-reduction steps vary slightly by band). Check with `src.get_gain_range(0)` as we did for the RSP1B; adjust the `sdrplay` entry in `SOAPY_DEFAULTS` only if it differs.

Windows build-from-source recipe (verified on Win 11, SDRplay API 3.15, Radioconda SoapySDR 0.8.1, VS 2022 Community 17.9, CMake 3.26):

```
# 1. Clone (Xilinx Vivado ships an old `git.exe` with a broken CA bundle on PATH —
#    use the system git if your shell finds the Vivado one first):
& "C:\Program Files\Git\cmd\git.exe" `
    -c http.sslbackend=schannel `
    clone --depth 1 https://github.com/pothosware/SoapySDRPlay3.git `
    $env:TEMP\SoapySDRPlay3
```

```
# 2. Configure against Radioconda's SoapySDR and the SDRplay API:
& "C:\Program Files\CMake\bin\cmake.exe" `
    -S $env:TEMP\SoapySDRPlay3 -B $env:TEMP\SoapySDRPlay3\build `
    -G "Visual Studio 17 2022" -A x64 -DCMAKE_BUILD_TYPE=Release `
    -DSoapySDR_DIR="C:/ProgramData/radioconda/Library/cmake" `
    -DLIBSDRPLAY_INCLUDE_DIRS="C:/Program Files/SDRplay/API/inc" `
    -DLIBSDRPLAY_LIBRARIES="C:/Program Files/SDRplay/API/x64/sdrplay_api.lib"


# 3. Build:
& "C:\Program Files\CMake\bin\cmake.exe" `
    --build $env:TEMP\SoapySDRPlay3\build --config Release --parallel


# 4. Install (Admin PowerShell — writes under C:\ProgramData):
Copy-Item "$env:TEMP\SoapySDRPlay3\build\Release\sdrPlaySupport.dll" `
          "C:\ProgramData\radioconda\Library\lib\SoapySDR\modules0.8\" -Force


# 5. Enable + start the SDRplay API service (still Admin):
Set-Service SDRplayAPIService -StartupType Manual
Start-Service SDRplayAPIService
```

Verify with `SoapySDRUtil --find` — the RSP should appear with `driver=sdrplay`.

Why the SDRplay API DLL needs special handling: `sdrPlaySupport.dll` depends on `sdrplay_api.dll` (installed by the SDRplay API installer at `C:\Program Files\SDRplay\API\x64\`). That directory is **not** on the system PATH after the installer runs, so SoapySDR can't load the support module out-of-the-box. The app calls `os.add_dll_directory(...)` at startup to register that location with the Python DLL loader — see the top of `dses_workbench.py`. End users do not need to munge PATH themselves.

Linux/macOS: Most distributions ship SoapySDRPlay3 in their package manager (`soapysdr-module-sdrplay` on Debian/Ubuntu, Homebrew tap `pothosware/homebrew-pothos`). The install guide's §3A documents that path; only Windows requires the build-from-source recipe above.

7.5 Memory / notes worth knowing

A few project facts that don't fit elsewhere:

- The B210 USRP serial used during development is **3273A91**. New users with their own B210s get auto-detected; no source change needed (see §8 of the install guide).
- macOS users on Apple Silicon must install the **arm64** Radioconda build; mixing an x86_64 Radioconda with arm64 Qt produces confusing errors at startup.
- On Linux, Radioconda's udev rules must be activated for the B210 (see install guide §2.2). SDRPlay devices on Linux similarly need the udev rules from the SDRplay API installer.
- The auto-update check is harmless if the manifest URL is unreachable — failures are silent in the background path. Don't panic if you accidentally delete the manifest; users just don't see new-version notifications until you restore it.